

Disciplina: Programação Orientada à Objetos

Aluno(a): _____ Nota: _____

Prof. Tiago Pessoa Ferreira de Lima

1. Questão Aberta: Encapsulamento

Analise a classe ContaBancaria abaixo. Atualmente, o atributo saldo pode ser modificado para qualquer valor, o que pode causar inconsistências graves em um sistema financeiro.

```
class ContaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular
        self.saldo = saldo_inicial # Atributo vulnerável

    def depositar(self, valor):
        if valor > 0:
            self.saldo += valor
            print(f"Depósito de R$ {valor:.2f} realizado.")
        else:
            print("Valor de depósito inválido.")

# Exemplo de uso problemático
conta = ContaBancaria("João Silva", 1000.00)
conta.saldo = -500.00 # Modificação direta e inválida
print(f"Saldo atual: R$ {conta.saldo:.2f}")
```

Com base no código, responda:

- Explique qual é o problema de o atributo saldo poder ser modificado diretamente e dê um exemplo de consequência negativa em um sistema bancário.
- Reescreva a classe ContaBancaria utilizando a forma "Pythônica" de encapsulamento (@property e @<atributo>.setter) para o atributo saldo.
- A nova implementação deve garantir que o saldo nunca possa ser definido como um valor negativo. Se uma tentativa de atribuição negativa ocorrer, uma exceção ValueError com a mensagem "O saldo não pode ser negativo." deve ser levantada.

2. Questão Aberta: Herança e Polimorfismo

Considere a classe base Animal abaixo, que representa um animal de forma genérica.

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def emitir_som(self):
        print("O animal emite um som genérico.")
```

Com base no código, responda:

- a) Usando o conceito de Herança, explique por que criar classes Cachorro e Gato que herdam de Animal é mais vantajoso do que criá-las do zero.
- b) Crie as classes Cachorro e Gato como subclasses de Animal.
- c) O construtor de Cachorro deve aceitar nome e raça, e o de Gato deve aceitar nome e pelagem. Utilize a função `super()` para inicializar o atributo herdado.
- d) Sobrescreva (override) o método `emitir_som()` em cada subclasse para que Cachorro imprima "Au au!" e Gato imprima "Miau!".

Questões de Múltipla Escolha

- 3. O que melhor define o pilar da Abstração em Programação Orientada a Objetos?
 - A) A capacidade de uma classe herdar características de outra.
 - B) O processo de esconder os detalhes complexos de implementação e expor apenas as funcionalidades essenciais de um objeto.
 - C) A restrição do acesso direto aos dados de um objeto, forçando o uso de métodos.
 - D) A capacidade de um método ou objeto ter múltiplas formas.
 - E) A criação de um objeto a partir de uma classe.
- 4. Qual modificador de acesso torna um atributo ou método acessível apenas dentro da própria classe onde foi definido?
 - A) Público (public)
 - B) Protegido (protected)
 - C) Privado (private)
 - D) Global
 - E) Estático (static)
- 5. A prática de definir múltiplos métodos com o mesmo nome, mas com diferentes parâmetros (em número ou tipo) dentro da mesma classe, é conhecida como:
 - A) Sobrescrita (Override)
 - B) Herança Múltipla
 - C) Encapsulamento
 - D) Abstração
 - E) Sobrecarga (Overload)
- 6. Qual dos conceitos abaixo é fundamental para garantir que um objeto controle seu próprio estado, evitando modificações externas indesejadas?
 - A) Herança
 - B) Polimorfismo
 - C) Associação

D) Encapsulamento

E) Composição

7. Em um sistema de uma loja, as classes Produto, Cliente e Pedido se relacionam. Um Pedido contém uma lista de Produtos e está associado a um Cliente. Esse tipo de relacionamento, onde objetos de classes diferentes colaboram, é um exemplo de:

A) Herança

B) Associação

C) Polimorfismo

D) Abstração

E) Instanciação

Rascunho